

C++ and Object Oriented Programming



Unit – 5 (Topic – 1) Working With Files

**Prepared By
Prof. Shah Brijesh**

Working With Files

Q : File stream classes:

File Stream:

-> The I/O system of C++ contains a set of classes that define the file handling methods.

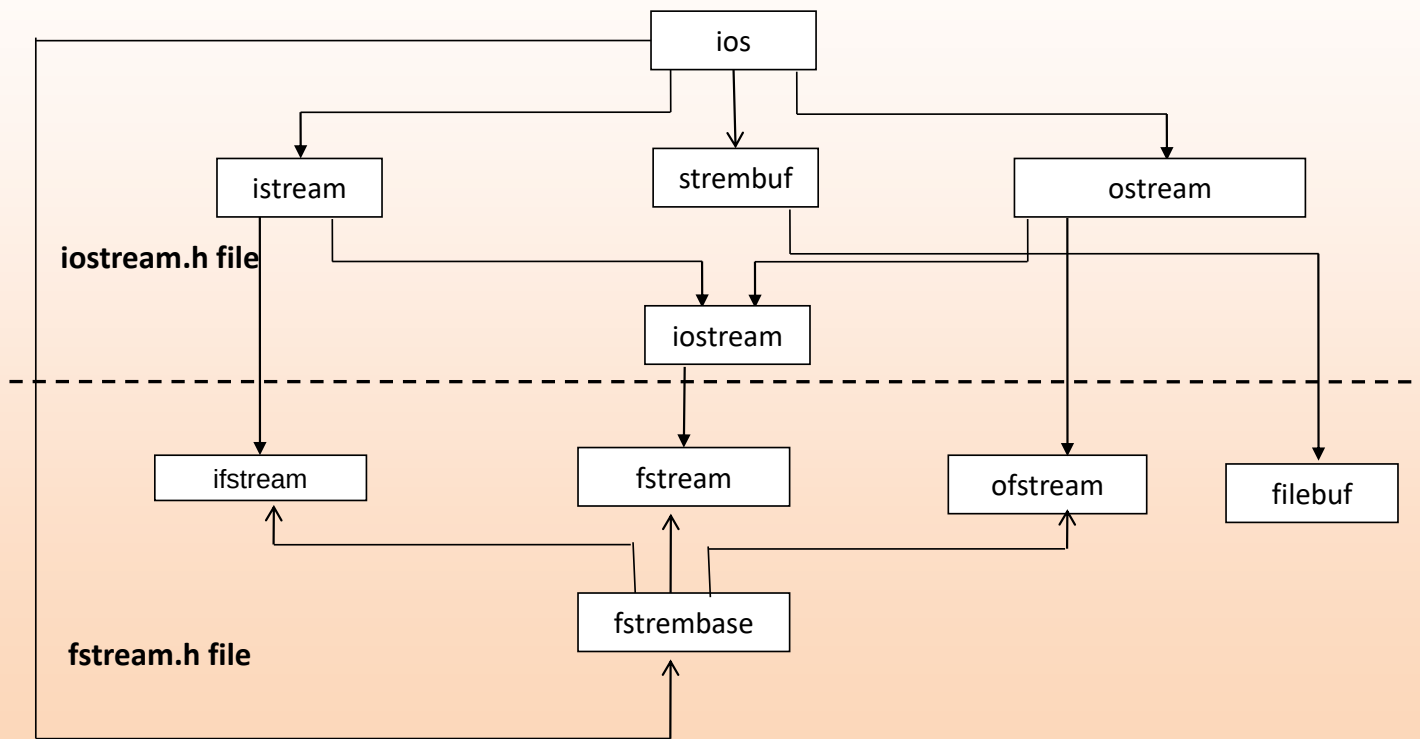
-> These include “ifstream”, “ofstream” and “fstream”.

-> These classes are derived from “fstreambase” and from the corresponding “iostream” class as shown in below figure.

-> These classes, designed to manage the disk files are declared in “fstream.h” header file and therefore we must include this file in any program that uses files.

Working With Files

Q : File Stream Classes:



Working With Files

Q : File Stream Classes:

1. filebuf:

- > It's purpose is to set the file buffers to read and write.
- > Contains openprot constant used in the open() of file stream classes.
- > Also contain close() and open() as members.

2. fstreambase:

- > Provides operations common to the file streams.
- > Serves as a base for fstream, ifstream and ofstream class.
- > Contains open() and close() functions.

3. ifstream:

- > Provides input operations

Working With Files

Q : File Stream Classes:

- > Contains open() with default input mode.
- > Inherits the functions get(), getline(), read(), seekg() and tellg() functions from istream.

4. ofstream:

- > Provides output operations common to the file streams.
- > Contains open() with default output mode.
- > Inherits put(), seekp(), tellp() and write() functions from ostream.

5. fstream:

- > Provides support for simultaneous input and output operations.
- > Contains open() with default input mode.
- > Inherits all the functions from istream and ostream classes through iostream.

Working With Files

Q : Opening a File:

- > A file must be opened before you can read from it or write to it.
- > Either **ofstream** or **fstream** object may be used to open a file for writing and ifstream object is used to open a file for reading purpose only.
- > A file can be opened in two ways.
 1. Using constructor function of the class.
 2. Using the member function open() of the class.

1. Opening Files Using Constructor:

- > We know that a constructor is used to initialize an object while it is being created.
- > Here a file name is used to initialize the file stream object.
- > This involves the following steps.
 - => Create a file stream object to manage the stream using the appropriate class.
 - => That is, the class ofstream is used to create the output stream and the class ifstream to create the input stream.
 - => Initialize the file object with the desired file name.

Working With Files

Q : Opening a File:

-> For Example, the following statement opens a file named “results” for output:

```
ofstream outfile(“results”);
```

-> This creates outfile as an ofstream object that manages the output stream.

-> This object can be any valid c++ name.

-> This statement also opens the file results and attaches it to the output stream outfile.

Note: Closing a File:

-> When a C++ program terminates it automatically flushes all the streams, release all the allocated memory and close all the opened files.

-> But it is always a good practice that a programmer should close all the opened files before program termination.

-> Following is the standard syntax for close() function, which is a member of fstream, ifstream, and ofstream objects.

```
void close();
```

Working With Files

Q : Opening a File:

```
void main()
{
    char name[20];
    int rn;
    ofstream outf("Demo1.txt");
    clrscr();
    cout<<"Enter Roll NO. & Name of a student";
    cin>>rn>>name;
    outf<<rn<<"\n";
    outf<<nm<<"\n";
    outf.close();
    ifstream inf("Demo1.txt");
    inf>>rn;
    inf>>name;
    cout<<"\n Roll no. ="<<rn<<"\n name ="<<nm;
    inf.close();
    getch();
}
```

Working With Files

Q : Opening a File:

2. Opening File Using open():

-> The function open() can be used to open multiple files that use the same stream object.

-> For example, we may want to process a set of files sequentially.

-> For example, we may want to process a set of files sequentially.

-> In such cases, we may create a single stream object and use it to open each file in turn.

-> This is done as follows:

```
file-stream-class stream-object;
```

```
stream-object.open("filename");
```

-> Following Program demonstrate how we can open more than one file in a single stream object.

Working With Files

Q : Opening a File:

```
void main()
{
    char name[20];
    clrscr();
    ofstream outf;
    outf.open("country.txt");
    outf<<"India\n"<<"USA\n"<<"UK\n";
    outf.close();
    outf.open("capital.txt");
    outf<<"New Delhi\n"<<"Washington\n"<<"London\n";
    outf.close();
    ifstream inf;
    inf.open("country.txt");
    cout<<"\n Content Of Country.txt File \n";
    while(inf)
    {
        inf.getline(name, 19);
        cout<<name<<endl;
    }
}
```

Working With Files

Q : Opening a File:

```
inf.close();
inf.open("capital.txt");
cout<<"\n Content Of Capital.txt File \n";
while(inf)
{
    inf.getline(name, 19);
    cout<<name<<endl;
}
inf.close();
getch();
}
```

More about open():

-> open() can take two arguments, the second argumentspecifying the file mode.

-> The general form of the open() function with argument is:

```
stream-object.open("filename", mode);
```

Working With Files

Q : Opening a File:

- > The second argument mode specifies the purpose for which the file is opened.
- > The file mode parameter can take one (or more) of such constants defined in the class **ios**.
- > The following table lists the file mode parameters and their meaning.

Parameter	Meaning
<code>ios::app</code>	Append to end of file
<code>ios::ate</code>	Go to end of file on opening
<code>ios::binary</code>	Binary File
<code>ios::in</code>	Open file for reading only
<code>ios::nocreate</code>	Open fails if the file does not exist.
<code>ios::noreplace</code>	Open files if the file already exist
<code>ios::out</code>	Open file for writing only
<code>ios::trunc</code>	Delete the contents of the file if it exist

Notes:

1. Opening a file in `ios::out` mode also opens it in the `ios::trunc` mode by default.
2. The `fstream` class does not provide a mode by default and therefore, we must provide the mode explicitly when using an object of `fstream` class.
3. The mode can combine two or more parameters using the bitwise OR operator
`fout.open("demo1.txt", ios::out | ios::in);`

Working With Files

Q : Opening a File:

```
void main()
{
    char line[80];
    clrscr();
    fstream f1;
    f1.open("demo2.txt", ios::in | ios::out);
    cout<<"Enter a line of text:";
    cin.getline(line,79);
    for(int i=0; i<strlen(line; i++)
        f1.put(line[i]);
    f1.seekg(0);
    char ch;
    cout<<"\n Content Of the file Demo2.txt\n";
    while(f1)
    {
        f1.get(ch);
        cout<<ch;
    }
    f1.close();
    getch();
}
```

Working With Files

Q : Detecting End Of File:

-> Detecting of the end of file condition is necessary for preventing any further attempt to read data from the file.

-> The statement

```
while(fin)
```

-> An ifstream object fin, returns a value of 0 if any error occurs in the file operation including the end of file condition.

-> Thus, the while loop terminates when fin returns a value of zero on reaching the end of file condition.

Note : This loop may terminate due to other failures as well.

-> There is another way to detect the end of file condition.

-> The statement

```
if(fin.eof() != 0)
{
    exit(0);
}
```

-> eof() is a member function of ios class.

-> It returns a non-zero value if the end of file condition is encountered and a zero otherwise.

-> Therefore, the above statement terminates the program on reaching the end of the file.

Working With Files

Q : File Pointers & Their manipulation:

- > Each file has two associated pointers known as file pointers.
- > One of them is called input pointer (get pointer) and the other is called output pointer (put pointer).
- > We can use these pointers to move through the files while reading or writing.
- > The input pointer is used for reading the contents of a given file location and the output pointer is used for writing to a given file location.
- > Each time an input or output operation takes place, the appropriate pointer is automatically advanced.
- > Default Action :
 1. When we open a file in read only mode, the input pointer is automatically set at the beginning of the file, so that we can read the file from the start.
 2. When we open a file in write only mode, the existing contents are deleted and the output pointer is set at the beginning of the file. This enables us to write to the file from the start.
 3. When we want to open an existing file to add more data, the file is opened in append mode. This moves the output pointer to the end of the file.

Working With Files

Q : File Pointers & Their manipulation:

Functions For Manipulation Of File Pointers:

-> All the actions on the file pointers take place automatically by default.
-> If we want to control the movement of the file pointers, the file stream object classes support the following functions to manage such situation.

1. `seekg()` : Moves get pointer (input) to a specified location
2. `seekp()` : Moves put pointer (output) to a specified location
3. `tellg()` : Gives the current position of the get pointer.
4. `tellp()` : Gives the current position of the put pointer.

Syntax:

```
tellp();  
tellg();  
seekg(offset, reposition);  
seekp(offset, reposition);
```

Description:

-> Here offset represents the number of bytes the file pointer is to be moved from the location specified by the parameter reposition.

Working With Files

Q : File Pointers & Their manipulation:

-> The reposition takes one of the following three constants defined in the ios class.

ios::beg	Start of the file
ios::cur	Current position of the pointer
ios::end	End of the file

-> The seekg() function moves the associated file's get pointer while the seekp() function moves the associated file's put pointer.

-> Following table lists some sample pointer offset calls and their action. (Here fout is an ofstream object).

Seek call	Action
fout.seekp(0, ios::beg)	Go to start
fout.seekp(0, ios::cur)	Stay at the current position
fout.seekp(0, ios::end)	Go to the end of the file
fout.seekp(m, ios::beg)	Moves to (m+1)th byte in the file
fout.seekp(m, ios::cur)	Go forward by m byte from the current position
fout.seekp(-m, ios::cur)	Go backward by m byte from the current position
fout.seekp(-m, ios::end)	Go backward by m byte from the end

Working With Files

Q : Sequential Input and Output Operations:

-> The file stream classes support a number of member functions for performing the input and output operations on files.

-> One pair of functions, `put()` and `get()` are designed for handling a single character at a time.

-> Another pair of functions `write()` and `read()` are designed to write and read blocks of binary data.

put() and get() Functions:

```
void main()
```

```
{
```

```
    char c;
```

```
    clrscr();
```

```
    cout<<"Enter content of the file (Ctrl+z To Terminate)\n";
```

```
    fstream f;
```

```
    f.open("putget.txt",ios::in | ios::out);
```

```
    c=cin.get();
```

Working With Files

Q : Opening a File:

```
while(c!=EOF)
{
    f.put(c);
    c= cin.get();
}
f.seekg(0);
cout<<"\n Content Of the file \n";
while(f)
{
    f.get(c);
    cout<<c;
}
f.close();
getch();
}
```

Working With Files

Q : Sequential Input and Output Operations:

write() and read() Functions:

- > The functions write() and read(), unlike the functions put() and get(), handle the data in binary form.
- > This means that the values are stored in the disk file in the same format in which they are stored in the internal memory.
- > The binary format is more accurate for storing the numbers as they are stored in the exact internal representation.
- > There are no conversions while saving the data and therefore saving is much faster.
- > The binary input and output functions take the following form:
 infile.read((char *) & V, sizeof(V));
 outfile.write((char *) & V, sizeof(V));
- > These functions take two arguments.
- > The first is the address of the variable V, and the second is the length of that variable in bytes.
- > The address of the variable must be cast to type char * (i.e. pointer to character type)

Working With Files

Q : Sequential Input and Output Operations:

write() and read() Functions:

```
void main()
{
    float f[] = {12.3, 23.4, 34.5, 45.6};
    clrscr    ();
    fstream f1;
    f1.open("rdwrt1.txt", ios::in | ios::out);
    f1.write((char*) &f, sizeof(f));
    f1.seekg(0);
    for(int i=0; i<4; i++)
        f[i] = 0;
    f1.read((char*) &f, sizeof(f));

    for(i=0; i<4; i++)
        cout<<endl<<f[i];
    getch();
}
```

Working With Files

Q : Sequential Input and Output Operations:

Reading & Writing Class Object :

```
class Inventory
{
    char nm[30];
    int code;
    float cost;
    public:
        void get();
        void put();
};

void Inventory :: get()
{
    cout<<"Enter name:"; cin>>nm;
    cout<<"Enter code:"; cin>>code;
    cout<<"Enter Cost:"; cin>>cost;
}

void Inventory ::put()
{
    cout<<"\n Name ="<<nm;
    cout<<"\n Code ="<<code;
    cout<<"\n Cost ="<<cost;
}
```

Console Input / Output Operations

Q : Sequential Input and Output Operations:

Reading & Writing Class Object :

void main()

```
{
    Inventory i1;
    char c;
    clrscr();
    fstream f1;
    f1.open("rdwrt2.txt", ios::in | ios::out);
    while(1)
    {
        i1.get();
        f1.write((char*) &i1, sizeof(i1));
        cout<<"\n Continue...?";
        cin>>c;
        if(c!='y' )
            break;
    }
    f1.seekg(0);
    while(f1.read((char*) &i1, sizeof(i1)))
    {
        i1.put();
    }
    f1.close();           getch();           }
```

Working With Files

Q: Updating A File : Random Access:

- > Updating is a routine task in the maintenance of any data file.
- > The updating would include one or more of the following task.
 - > Displaying the contents of a file.
 - > Modifying an existing item.
 - > Adding a new item.
 - > Deleting an existing item.
- > These actions require the file pointers to move to a particular location that corresponds to the item/object under consideration.
- > This can be easily implemented if the file contains a collection of items/objects of equal length.
- > In such cases, the size of each object can be obtained using the statement:

```
int obj_len = sizeof (object);
```

- > Then, the location of a desired object, say the mth object, may be obtained as follows:

```
int location = m * obj_len;
```

Working With Files

Q : Updating A File : Random Access:

-> The location gives the byte number of the first byte of the mth object.

-> Now, we can set the file pointer to reach this byte with the help of seekg() or seekp().

-> We can also find out the total number of objects in a file using the obj_len as follows:

```
int n = file_size / obj_len;
```

-> The file_size can be obtained using the function tellg() or tellp() when the file pointer is located at the end of the file.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<fstream.h>
```

```
class Inventory
```

```
{    char nm[30];
```

```
    int code;
```

```
    float cost;
```

```
    public:
```

```
        void get();
```

```
        void put();
```

```
};
```

Working With Files

Q : Updating A File : Random Access:

```
void Inventory :: get()
{
    cout<<"\nEnter name:";   cin>>nm;
    cout<<"Enter code:";     cin>>code;
    cout<<"Enter Cost:";     cin>>cost;
}
void Inventory ::put()
{
    cout<<"\n"<<nm;
    cout<<"\t"<<code;
    cout<<"\t"<<cost;
}
void main()
{
    Inventory i1;
    char c;
    clrscr();
    fstream f1;
    f1.open("rdwrt2.txt", ios::in | ios::out | ios::ate | ios::binary);
    f1.seekg(0);
    cout<<"\n CURRENT CONTENTS OF STOCK \n";
    cout<<"\n Name \t Code \t Cost";
    while(f1.read((char*) &i1, sizeof(i1)))
    {
        i1.put();
    }
}
```

Working With Files

Q : Updating A File : Random Access:

```
f1.clear();
cout<<"\n Add An Item";
i1.get();
f1.write((char*) &i1, sizeof(i1));

cout<<"\n Appended Content Of Stock \n";
cout<<"\n Name \t Code \t Cost";
f1.seekg(0);
while(f1.read((char*) &i1, sizeof(i1)))
{
    i1.put();
}

int last = f1.tellg();
int n = last / sizeof(i1);

cout<<"\n Number Of Objects ="<<n;
cout<<"\n Total Bytes In File ="<<last;

cout<<"\n Enter Object No. To Modify:";
int objects;
cin>>objects;
cin.get(c);
int location = (objects-1) * sizeof(i1);

if(f1.eof())
    f1.clear();
```

Working With Files

Q : Updating A File : Random Access:

```
cout<<"\n Enter new values of the object :";
i1.get();
cin.get(c);
f1.seekp(location);
f1.write((char*) &i1, sizeof(i1));

f1.seekg(0);
cout<<"\n Content Of Updated File\n";
while(f1.read((char*) &i1, sizeof(i1)))
{
    i1.put();
}
f1.close();
getch();
}
```

Working With Files

Q : Error Handling During File Operations:

-> Whenever we open the file, one of the following things may happen.

1. A file which we are attempting to open for reading does not exist.
2. The file name used for a new file may already exist.
3. We may attempt an invalid operation such as reading past the end of file.
4. There may not be any space in the disk for storing more data.
5. We may use an invalid file name.
6. We may attempt to perform an operation when the file is not opened for that purpose.

-> The c++ file stream inherits a 'stream-state' member from the class ios.

-> This member records information on the status of a file that is being currently used.

-> The stream state member used bit fields to store the status of the error conditions stated above.

-> The class ios supports several member functions that can be used to read the status recorded in a file stream.

-> These functions along with their meanings are listed in below table.

Working With Files

Q : Error Handling During File Operations:

Functions	Return value and meaning
eof()	-> Returns true if end of file is encountered while reading. -> otherwise returns false.
fail()	-> Returns true when an input or output operation has failed.
bad()	-> Returns true if an invalid operation is attempted or any unrecoverable error has occurred. -> However, if it false, it may be possible to recover from any other error reported and continue operation.
good()	-> Returns true if no error has occurred. -> This means, all the above functions are false. -> For instance, if file.good() is true, all is well with the stream file and we can proceed to perform i/o operations. -> When it returns false, no further operations can be carried out.

Working With Files

Q : Command Line Arguments:

-> Like C, C++ too supports a feature that facilitated the supply of arguments to the main().

-> These arguments are supplied at the time of invoking the program.

-> They are typically used to pass the names of data files.

-> For Example:

C:\> Filename data results

-> Here Filename is the name of the file containing the program to be executed and data and results are the filenames passed to the program as command line arguments.

-> The command line arguments are typed by the user and are delimited by a space.

-> The first arguments always the filename (Command name) and contains the program to be executed.

-> For command line arguments main() takes two arguments as shown below.

main(int argc, char * argv[])

-> The first argument argc (known as argument counter) represents the number of arguments in the command line.

Working With Files

Q : Command Line Arguments:

-> The second argument argv (known as argument vector) is an array of char type pointers that points to the command line arguments.

-> The size of this array will be equal to the value of argc.

-> For instance, for command line

c:\>Filename data results

-> The value of argc would be 3 and argv would be an array of three pointers to strings as shown below.

argv[0] -> Filename

argv[1] -> data

argv[2] -> results

-> Note that argv[0] always represents the command name that invokes the program.

Working With Files

Q : Command Line Arguments:

Program that demonstrate copy of one file into another using command line arguments

```
void main(int argc, char * argv[])
{
    if(argc!=3)
    {
        cout<<"Invalid no. of arguments";
        exit(0);
    }
    ifstream f1(argv[1]);
    if(f1.fail())
    {
        cout<<'File Does not exist";
        exit(0);
    }
    ofstream f2(argv[2]);
    if(f2.fail())
    {
        cout<<"Insufficient disk space";
        exit(1);
    }
}
```

Working With Files

Q : Command Line Arguments:

Program that demonstrate copy of one file into another using command line arguments

```
char c;
while(1)
{
    c = f1.get();
    if(f1.eof())
        break;
    f2.put(c);
}
if(f1.good() && f2.good())
    cout<<"File Copied successfully";
}
```